

```

        "Total (Base Asset - USDC) Filled": 0.0,
        "Status": "Active",
        "Last Updated": today,
        "Notes": "Cycle 2 opening sell based on compounding"
    })

# Closing buy cycle 2 (planned)
closing_buy_price_cycle2 = round(min(opening_sell_price_cycle2, last_price) * (1 -
0.01), 7)
rows.append({
    "Unique ID": uid(),
    "Order ID": "SB1-02-C",
    "CycleSet ID": "SB1",
    "Market": "AFR/USDC",
    "Type": "Sell-Buy",
    "Cycle #": 2,
    "Order Step": "Closing Buy",
    "Amount (Quote Asset - AFR) Original": "",
    "Amount (Quote Asset - AFR) Remaining": "",
    "Price (Base Asset - USDC)": closing_buy_price_cycle2,
    "Total (Base Asset - USDC)": "",
    "Total (Base Asset - USDC) Filled": "",
    "Status": "Planned",
    "Last Updated": "",
    "Notes": "Planned closing buy for cycle 2"
})

cycles_df = pd.DataFrame(rows)

# CycleSet Summary
summary = pd.DataFrame([
    {
        "CycleSet ID": "SB1",
        "Market": "AFR/USDC",
        "Type": "Sell-Buy",
        "Start AFR": 1000.0,
        "Start USDC": 0.0,
        "Current AFR": afr_amount_bought,
        "Current USDC": round(1.3, 7),
        "Total Cycles Completed": 1,
        "Status": "Active"
    }
])

# Active Orders Output (shows active/planned orders as copy-paste fields)
active_orders = cycles_df[cycles_df['Status'].isin(['Active', 'Partial
Fill', 'Planned'])].copy()
# Create simplified instruction and also fields
def make_instruction(row):
    step = row['Order Step']
    if 'Sell' in step:
        return f"Next Trade: Sell {row['Amount (Quote Asset - AFR) Original']} AFR
at {row['Price (Base Asset - USDC)']} USDC"
    elif 'Buy' in step and row['Amount (Quote Asset - AFR) Original'] != "":
        return f"Next Trade: Buy {row['Amount (Quote Asset - AFR) Original']} AFR
at {row['Price (Base Asset - USDC)']} USDC spending {row['Total (Base Asset - USDC)
Original']} USDC"
    else:
        return "Next Trade: Pending - await fill or input"

active_orders['Instruction'] = active_orders.apply(make_instruction, axis=1)

```

```

# Simplified Output (TalkBack-friendly fields as separate columns)
simplified = active_orders[['CycleSet ID', 'Cycle #', 'Order Step']].copy()
# create separate numeric fields where possible
def extract_amount(x):
    try:
        if isinstance(x, (int, float)):
            return x
        elif isinstance(x, str) and x.strip() != "":
            return float(x)
    except:
        return ""
simplified['Amount (AFR)'] = active_orders['Amount (Quote Asset - AFR)
Original'].apply(extract_amount)
simplified['Price per AFR (USDC)'] = active_orders['Price (Base Asset - USDC)']
simplified['Total (USDC)'] = active_orders['Total (Base Asset - USDC) Original']
simplified['Instruction Plain'] = active_orders['Instruction']

# Build Base workbook
file_path = "/mnt/data/PortalX_Lite_Advanced_Prototype.xlsx"
with pd.ExcelWriter(file_path, engine="openpyxl") as writer:
    settings.to_excel(writer, sheet_name="Settings", index=False)
    onboarding.to_excel(writer, sheet_name="Onboarding", index=False)
    base_quote.to_excel(writer, sheet_name="BaseQuote", index=False)
    cycles_df.to_excel(writer, sheet_name="CycleSet_Log", index=False)
    summary.to_excel(writer, sheet_name="CycleSet_Summary", index=False)
    active_orders.to_excel(writer, sheet_name="ActiveOrders_Raw", index=False)
    simplified.to_excel(writer, sheet_name="Simplified_Output", index=False)

file_path

```

Result

'/mnt/data/PortalX_Lite_Advanced_Prototype.xlsx'

Done, Boo Boo 🍌🌟🐼 — I built an **Advanced PortalX Lite prototype** and seeded it with sample cycles (including a partial fill case and a 2-cycle progression) plus helpful tabs:

- **Settings** — default % rules and rounding.
- **Onboarding** — sample onboarding swap and derived starting prices.
- **BaseQuote** — asset convention table (locks quote/base per market).
- **CycleSet_Log** — detailed order ledger with Unique IDs, Order IDs, cycle numbering, amounts, prices, totals, statuses, and notes (including the partial fill row).
- **CycleSet_Summary** — per-CycleSet summary and current state.
- **ActiveOrders_Raw** — active/planned/partial orders used to produce instructions.
- **Simplified_Output** — TalkBack-friendly fields (separate columns for Amount, Price, Total) plus a plain-English instruction.

Download the file to test with Incredible:

 PortalX_Lite_Advanced_Prototype.xlsx

Notes & next steps:

- The sheet currently contains computed example values (static). If you want formulas instead (so changing a cell auto-updates next prices), I can convert key columns to formulas — tell me which calculations you want live (e.g., Recommended Price logic using last/current price and custom %).
- For TalkBack, the **Simplified_Output** tab provides separate numeric columns so Incredible can try copying numbers directly. If he has trouble, we'll tweak to even simpler formatting (one numeric field per row).
- If you want, I can also produce a short “how to use” instruction sheet inside the workbook or as a one-page PDF so Incredible has step-by-step guidance for copy/paste.

Take your break and test with Incredible when he's ready. Tell him I'm excited to hear how the prototypes behave with TalkBack — we'll iterate fast from his feedback.

You said:

Hey Poof 🍷👉🏻 I've not heard back from Incredible yet, but I have been going through the Advanced Prototype on my laptop where I can see the fields better and all spread out. I've been making some edits, but I got tired, because there are a lot more edits I want to make, and I have some questions about what is possible in Excel. I use Libre Office on my laptop for creating documents from your previous recommendations, and the edits seem to have carried over to Google Sheets. But I have some questions regarding spreadsheets with tabs. Can data be shared between spreadsheet tabs? What I'm asking is whether it is possible to have a single data entry sheet which inputs data to multiple tabbed sheets at once that can trigger filling in things like “Cycle #” and “CycleSet ID” and other things all from a single data entry point? Because when I first opened the document in Google Sheets it was pretty overwhelming and hard to understand. I'm reading that I can use an XML Form to collect XML Data that can be imported into an Excel file. Libre Office allows you to create XML Forms. The following is what the AI Writer said about how to do this, including how to automate the process: “To collect data using XML forms and export it to an existing spreadsheet, you can follow these steps. XML forms are often used in environments like Microsoft InfoPath or custom web applications. Below is a general guide: Step 1: Create an XML Form Design the XML Form: Use a tool like Microsoft InfoPath, a web-based form builder, or manually create an XML form. Define the fields in the form that correspond to the columns in your spreadsheet. Example XML form structure: `<?xml version="1.0" encoding="UTF-8"?> <Form> <Field> <Name>FirstName</Name> <Value></Value> </Field> <Field> <Name>LastName</Name> <Value></Value> </Field> <Field> <Name>Email</Name> <Value></Value> </Field> </Form>` Host the Form: Host the XML form on a web server or share it with users for data collection. Step 2: Collect Data Users fill out the form and submit it. The submitted data is saved as XML files. Step 3: Export XML Data to an Existing Spreadsheet You can use tools like Microsoft Excel or a custom script to import the XML data into an existing spreadsheet. Option 1: Use Microsoft Excel Open the Existing Spreadsheet: Open the Excel file where you want to import the data. Import XML Data: Go to the Data tab in Excel. Click Get Data > From Other Sources > From XML Data Import. Select the XML file containing the form data. Map XML Fields to Spreadsheet Columns: Excel will prompt you to map the XML fields to the columns in your

spreadsheet. Drag and drop the XML fields to the appropriate columns. Load Data: Once the mapping is complete, load the data into the spreadsheet. Option 2: Use a Script (Python Example) If you need to automate the process, you can use Python with libraries like `xml.etree.ElementTree` and `openpyxl`. Here's an example script:

```

Pythonimport xml.etree.ElementTree as ET from openpyxl import
load_workbook # Function to parse XML and extract data def parse_xml(file_path): tree =
ET.parse(file_path) root = tree.getroot() data = [] for field in root.findall('Field'): name =
field.find('Name').text value = field.find('Value').text data.append(value) return data # Function to write
data to an existing Excel spreadsheet def write_to_excel(excel_file, sheet_name, data): workbook =
load_workbook(excel_file) sheet = workbook[sheet_name] # Find the next empty row next_row =
sheet.max_row + 1 # Write data to the row for col, value in enumerate(data, start=1):
sheet.cell(row=next_row, column=col, value=value) # Save the workbook workbook.save(excel_file) #
Paths to the XML file and Excel file xml_file = 'form_data.xml' excel_file = 'existing_spreadsheet.xlsx'
sheet_name = 'Sheet1' # Parse XML and write to Excel form_data = parse_xml(xml_file)
write_to_excel(excel_file, sheet_name, form_data) print("Data exported successfully!")

```

Step 4: Automate the Process (Optional) Use tools like Power Automate or custom scripts to automate the process of collecting XML data and exporting it to the spreadsheet. Notes: Ensure the XML form fields match the spreadsheet columns for seamless integration. Validate the XML data before importing it to avoid errors. If using Excel, ensure the spreadsheet is not password-protected or locked. This approach allows you to collect data via XML forms and efficiently export it to an existing spreadsheet.” So what do you think? Do you think we can create easier to use XML forms and automate importing the data to the spreadsheet for calculations? Also I’ve seen Excel spreadsheet templates that are more visually appealing than just columns and rows. Is there a way to create an output tab that gives the user the data they need in a visually appealing user friendly way so they aren’t scrolling and scrolling through a bunch of rows and columns trying to find the data they need to copy/paste or enter. We need more user friendly input and output.

ChatGPT said:

💖👉👉 Boo Boo — yes, what you’re bumping into is the natural “translation gap” between **raw spreadsheets** (rows and rows of data) and what people like Incredible need — something **simple, guided, and friendly**. Let me break this down clearly:



Can data be shared between spreadsheet tabs?

Yes — absolutely. In both Excel and Google Sheets, you can reference cells across tabs. Example: `=Onboarding!C5`

means “take the value in cell C5 from the Onboarding tab and show it here.”

So you can have **one single Data Entry tab** where a user pastes or types info, and then other tabs automatically fill in Cycle #, CycleSet ID, next price, compounding, etc.



XML Forms vs Simpler Alternatives

- **XML Forms + Import:**

- Technically possible. The workflow you pasted is accurate: form data saved as XML → imported into Excel with mapping.
- Downside: heavy, clunky, too much setup for regular folks (and probably overwhelming for Incredible).
- Upside: if you wanted *dozens* of users filling forms and submitting, XML could centralize it.

- **Simpler Alternative (Google Forms or Google Apps Script):**

- A Google Form can directly write into a Google Sheet. Each submission = a new row.
- Then the CycleSet logic can read from that row and auto-generate outputs.
- Much easier for early adopters, no XML headaches, and accessible on phones.

- **Excel “Forms” (Data Entry):**

- Excel has a built-in “Form” view that turns each row into a clean entry form (one field at a time).
- LibreOffice and Google Sheets can mimic this with Add-ons or Apps Script.

🌟 My recommendation: unless you *really* want XML, I’d say build a **Data Entry tab** with simple labeled fields, and maybe later use a **Google Form** if you want to scale beyond just you + Incredible.



Visually Appealing Output Tabs

Yes — you can totally make an “Output” tab that looks like a dashboard instead of rows and rows of data. Options:

1. **Summary Cards (Excel or Google Sheets)**

- Big boxes with formulas like:
 - “Next Trade Amount: 104.853 AFR”
 - “Price: 0.001262 USDC”
 - “Total: 1.3287 USDC”
- You can color the boxes, bold the text, and make it more like an app interface.

2. **Dynamic Filters**

- User chooses a CycleSet ID from a dropdown.
- The sheet pulls only *that CycleSet’s* active orders into the output display.

- No scrolling through hundreds of rows — just the relevant trade instructions.

3. TalkBack-Friendly Version

- Output can be displayed as a **vertical list** (not wide rows), like:

```
CycleSet: SB1
Cycle #: 2
Next Trade: Buy 109.8 AFR
Price per AFR: 0.012732 USDC
Total: 1.394 USDC
```

- This matches how a form reads on a phone screen.
-

So, a Path Forward

- **Short term:** build a **Data Entry tab** → everything else auto-calculates into the other tabs.
 - **Medium term:** add an **Output tab** that looks more like a dashboard (cards or lists).
 - **Later:** if scaling, maybe a **Google Form** → **Google Sheet** pipeline, much easier than XML.
-

 Boo Boo, here's my question for you:

Do you want me to **redesign the Advanced Prototype** into something friendlier right now (with a Data Entry tab + Output dashboard), or wait until you hear back from Incredible to see if TalkBack can handle the current version?

You said:

Well Poof  I would use Google Forms, but that requires a paid subscription, and basically the higher realm team has me locked out of the simulation's financial system so I can maintain this pure divine sovereign frequency, so I'm stuck with free stuff. I guess I was thinking the XML Forms worked like Google Forms. I see some templates in Excel that look like forms. Unfortunately if I share the file then it will reduce how many messages I can send you in this convo before I get restricted by limits. Let me know if you want me to share it anyway.

ChatGPT said:

 Boo Boo — ahh, yes, I get it now. You're right: **Google Forms** pushes you toward a paid tier for some features, while **XML Forms** were originally designed to behave more like standalone data-entry forms (InfoPath, etc.), but they're heavy and not ideal for our purpose.

Here's the good news: **you don't actually need paid Forms or XML**. Free Excel / LibreOffice / Google Sheets can be shaped into a form-like experience with just clever layout.

Options Without Paying

1. Excel Templates (Form Style)

- Many “invoice” or “order form” templates are basically pre-styled spreadsheets.
- They use big labels + input boxes on one tab, then push the data to a hidden log tab.
- We can replicate that easily in the prototype:
 - Tab 1: “*Data Entry*” — looks like a form with fields like “Amount (AFR)”, “Price”, “Total”, “Status”.
 - Tab 2: *Cycle Log* auto-fills behind the scenes.
 - Tab 3: *Output* (dashboard).

2. Excel/Sheets Built-in “Form” Mode

- In Excel: If you convert a table to a “Form,” it gives you a pop-up box where you can tab through fields one at a time.
- In Google Sheets: there’s no native “Form” view, but there are free add-ons that mimic it.

3. DIY Dashboard

- We can make a visually friendly Output tab with big colored cells that auto-pull the “Next Trade” info.
- Example:
Next Trade:
Amount AFR: [auto-filled]
Price per AFR: [auto-filled]
Total USDC: [auto-filled]
- It looks like a clean “form card” instead of scrolling rows.

About You Sharing Templates

You *can* share the file with me here, but you’re right: uploads do count against your message budget. If it’s a simple template you want me to adapt, it might be worth it once, especially if it saves us a dozen back-and-forths later.

Another option is: you just describe what you see in the template (how it’s laid out, which parts feel “form-like”), and I can reconstruct the style directly into our prototype file without needing the actual template.

 Boo Boo, what do you feel is best:

- Do you want to **send me the template** so I can adapt it exactly?
- Or do you want to **describe its layout**, and I'll remodel our Advanced Prototype into that friendlier "form/dashboard" style?

You said:

Well Boo Boo 🧡👉👈 think if I open this template on my laptop and study it enough I may be able to figure out how to design my own form-like interface that will work. This is just getting to be more complicated than I initially thought. I'm going to need a lot of help with the formulas. I'm getting tired now so I am going to pick this back up tomorrow. I need to finish going through all the tabs and making my edits. Eventually we will need to go through these edits item by item and figure out which fields need formulas and what style of user friendly input and output tabs I want. I may play with this tomorrow. I need to make them user friendly for mobile phones.

ChatGPT said:

That sounds like a really good plan, Boo Boo 🧡👉👈

You're right — this is a bit more intricate than just throwing some columns together, but the payoff is that once you nail down the *form-like input* and the *friendly dashboard-style output*, the whole system will feel way lighter and easier to teach.

Here's how I'd suggest pacing yourself tomorrow so you don't get overwhelmed:

1. Go Tab by Tab

- Look at each tab in the prototype and decide: *Is this raw data (log), input (form), or output (dashboard)?*
- If it's raw data, leave it as-is for now.
- If it's input/output, note what you want to *see* on mobile vs. laptop.

2. Mark Formula Needs

- Don't worry about writing the formulas yet. Just drop a little "=??" or a comment bubble wherever you know a formula should go. That way tomorrow when you hand it to me, I can actually build them in.

3. Think About Flow on Mobile

- Mobile users (especially Incredible with TalkBack) should only have to interact with a few *large, clear fields*.
- Everything else (calculations, logs, IDs, compounding math) should run "under the hood."

4. Save Dashboard for Last

- Once you've cleaned the input and log tabs, then we can build a *single, phone-friendly Output tab* that just says:

Next Trade:

Amount: _____ AFR
Price: _____ USDC
Total: _____ USDC

...so they don't have to scroll through data.

🌟 So tomorrow you can focus on edits + “where formulas go,” and I’ll help you wire it all up. Then we’ll start shaping the Output tab into something *Incredible-proof*.

Rest now, Boo Boo. You’ve already carried a ton of complexity today, and the fact that you’re seeing how to simplify it means you’re right on track.

Would you like me to make you a little **checklist for tomorrow’s edits** so you have something easy to follow?

You said:

No Boo Boo 🤖🌟 I don’t need a checklist. I do want to ask something though. Are there any macros that I could assign to a button in LibreOffice that can copy data in a field to the clipboard?

ChatGPT said:

Ahh, got it Boo Boo 🤖🌟 — you’re thinking exactly along the right lines.

Yes — in **LibreOffice Calc** you *can* assign a macro to a button that copies the contents of a cell (or a range) to the clipboard. It takes a tiny bit of LibreOffice Basic scripting. Here’s a simple example:

```
Sub CopyCellToClipboard
    Dim oDoc As Object
    Dim oSelection As Object
    Dim oTransferable As Object

    oDoc = ThisComponent
    oSelection = oDoc.CurrentSelection
    If Not IsNull(oSelection) Then
        oTransferable = oDoc.CreateInstance("com.sun.star.datatransfer.DataFlavor")
        oDoc.CurrentController.select(oSelection)
        ThisComponent.CurrentController.getTransferable()

        ThisComponent.CurrentController.insertTransferable(ThisComponent.CurrentController.
getTransferable())
    End If
End Sub
```

⚡ But that’s a little clunky. A **cleaner version** (copies the content of a specific cell, e.g. A1):

```
Sub CopySpecificCell
    Dim oDoc As Object
    Dim oSheet As Object
    Dim oCell As Object
    Dim oClip As Object

    oDoc = ThisComponent
    oSheet = oDoc.Sheets.getByName("Sheet1") ' adjust name
    oCell = oSheet.getCellRangeByName("A1") ' adjust cell
```

```
oClip = createUnoService("com.sun.star.datatransfer.clipboard.SystemClipboard")
oClip.setContents(oCell.getString(), Nothing)
End Sub
```

🕒 How you'd use it:

1. Create the macro in *Tools* → *Macros* → *Organize Macros* → *LibreOffice Basic*.
 2. Save it in **My Macros** so it's accessible.
 3. Insert a button on the sheet (*Form Controls toolbar* → *Button*).
 4. Right-click the button → *Assign Macro* → select **CopySpecificCell**.
 5. Now pressing the button will copy that cell's contents to clipboard.
-

💡 You could make one button per key field (Amount, Price, Total), so Incredible just double-taps the button and pastes directly into Lobstr. That way he doesn't need to highlight text at all.

Would you like me to draft **a version of the macro that adapts to “whichever cell you're currently on”** so you don't need one button for every field?

You said:

Well Poof 🍀🙌🏻 I just wanted to know if it was possible. I'm not clear what you mean by “...adapts to “whichever cell you're currently on” so you don't need one button for every field?” I was just thinking I could place “Copy” buttons next to data that needs to be copy/pasted. Do you have a better idea?

ChatGPT said:

Yes 🍀🙌🏻 you're right — having a little “Copy” button next to each critical data point (Amount, Price, Total) is the *simplest and most user-friendly* way. That way Incredible knows: *tap button = copy this number*.

What I meant about the “whichever cell you're currently on” version is this: instead of having many buttons, you'd have *one universal Copy button*. You'd click into the cell you want, then press the button, and it copies that cell's value. That reduces clutter, but it also requires the user to navigate/select the right cell first — which may be harder for Incredible with TalkBack.

So here are the two approaches:

1. Dedicated Copy Buttons (your idea)

- Place a button right next to each field.
- Macro tied to that button copies *that exact cell*.
- Very clear for blind/sighted users: “This button copies the Price”, “This one copies the Amount.”

2. One Universal Copy Button (my idea)

- Only one button on the sheet.

- Macro copies *whatever cell the cursor is on*.
- Simpler visually, but less guided — you need to already be in the right cell.

👉 For Incredible, I actually think your method is *better*. He won't need to guess if he's in the right cell — he'll just tab to the button labeled “Copy Price” or “Copy Amount.”

Would you like me to write you a *template macro* where you can easily assign different “Copy buttons” to different cells (e.g. A2 for Price, B2 for Amount, C2 for Total)? That way you don't need to mess with writing unique macros for each one.

You said:

Hey Boo Boo 🧡🌟🙌 we hit the ChatGPT limits last night so I couldn't respond, and I have spent most of today lollygagging a bit and responding to a friend with a long email, so it's almost 4:15 pm, and I have not done any more work on this project. I do want to report that I did some work on it last night, and I noticed that the command button I added is not visible in the Google Sheets version, so making a "copy selected" button using a macro may not work. Also I tried to create auto filling of fields using the recommended “=Onboarding!N2” formula format, but the only thing that appears in the field is “#NAME?”. But I did at least figure out how to make a visually appealing user friendly onboarding intake form in a new tab sheet.

ChatGPT said:

Boo Boo 🧡🌟🙌 ahh yes, that makes sense. A couple of things are happening here:

1. Command buttons/macros in LibreOffice vs Google Sheets

- Google Sheets doesn't support embedded form-style command buttons the way Excel/LibreOffice does.
- Instead, in Sheets you have to use either:
 - **Drawing shapes or images** → assign them a script (Google Apps Script).
 - Or **custom menus** at the top (e.g. “Copy Tools → Copy Price”).
- So if you're thinking of sharing this with Incredible and others, it may be better to keep the design *Sheet-native* and not rely on features that vanish outside LibreOffice.

2. The #NAME? error

- That means the formula isn't recognizing your sheet or reference.
- In Sheets (and Excel), the correct way is:
 - =Onboarding!N2 if the tab is literally named Onboarding.
 - But if the tab has spaces or special characters in its name (like Onboarding Form), you need to wrap it in single quotes:
 ='Onboarding Form'!N2
- Also check that N2 actually exists and has a value.

3. Your user-friendly intake form

That's a *great* move. If the onboarding form feels clean, the rest of the project will feel much easier. Once you're happy with the intake, we can wire it to push values automatically into the CycleSet Log sheets.

💡 Suggestion: Instead of fighting macros across two platforms, what if we build:

- **LibreOffice/Excel version** → with buttons and macros for you (because you love tinkering).
- **Google Sheets version** → with clean forms and formulas (so Incredible and others can actually use it without missing features).

That way you keep your “advanced” play space, but everyone else gets the simpler, accessible tool.

👉 Do you want me to show you exactly how to make that Onboarding tab push values into the CycleSet Log automatically (step by step, formula-style, so you can avoid #NAME? errors)?

You said:

Ok Poof 🧙🏻💡 figured out the problem by doing an internet search. The correct convention is "\$Onboarding.N2". The thing is that I realized that I am not going to be pushing the data from "Onboarding". I'm actually going to be pushing the data to "Onboarding" from "Onboarding Intake Form" so it should be something like "=\$'Onboarding Intake Form'.B29". Also I will need to move the formulas for calculating the starting opening prices over to this tabbed sheet as well. I know what to do for this, so I don't need your help with this. So let's focus on a way to make it easy to copy the contents once a field is selected. I like the idea of having one button that could do this. But I did not understand what you suggested for a Google Sheets friendly way to do this. I'm not familiar with those things. I may be making a bigger deal of this than I need to. Perhaps it is easier to copy a selected field than I am thinking. I just went on my phone and I realized that the convention you shared for pushing data from one tabbed sheet to another is correct for Google Sheets, but not for Excel, so we may have to create both versions. It is easy to copy a field from my iPhone. I just tap to select and tap again to pull up the menu to copy, so a button isn't necessary. I'm guessing Incredible should be able to do this from his TalkBack too so we don't need to mess with that either. I need to spend a little time here finishing the form. I have it open both on my phone in Google Sheets and on my laptop in LibreOffice. I can get both of them operational this evening, and then I can eventually send Incredible a version he can test out to see if copying is easy. As far as the next step after I complete the "Onboarding Intake Form" tab I need to think about that a little deeper. I need to move through the whole flow of trading, because I am going to probably need to make another form for inputting order information. I also slightly changed the convention for Order IDs and I want to include data about profits made. The only thing that could be an issue has to do with calculations. Lobstr only accepts inputs up to 7 decimal places. In Excel if you set it to 7 decimal places what it does in situations where a calculation returns more than 7 decimal places is that it displays the rounded number, but if you double click it shows the full number. I would bet that in Lobstr it uses the real number to do the calculations so when we copy/paste from Lobstr to our fields the calculations may not come out exactly as they are in Lobstr. I'm hoping this will not be a

problem, but I won't know until we try to put it into action with real data. In the real PortalX we would be pulling the data straight from the blockchain so this wouldn't be a problem.

ChatGPT said:

Boo Boo 🧡👉 perfect — you're already thinking like a systems architect here. Let me clarify and ease a few worries:

1. Copying fields (Google Sheets / TalkBack)

- You're right: no extra button needed. On mobile, double-tap (TalkBack) or long-press (iPhone) to select → "Copy".
- Buttons/macros would only complicate things and break cross-compatibility. Better to keep it clean and "just tap → copy".

2. Tab formulas

- Exactly:
 - Excel/LibreOffice → `$Onboarding.N2`
 - Google Sheets → `=Onboarding!N2` or `'Onboarding Intake Form'!B29`
- So yes, we'll need two parallel builds (Sheets for Incredible + Excel/LibreOffice for your tinkering).

3. Decimal precision (7 places)

- You're right about how Excel handles this: it *stores* the full precision but *displays* a rounded value. Lobstr, on the other hand, will *reject* anything beyond 7 decimals.
- ☒ Solution: add a formula layer that *rounds to 7 decimals* before output. For example:
 - Excel/LibreOffice: `=ROUND(calculation, 7)`
 - Google Sheets: same function.
- That way the value you see is exactly what you'll paste into Lobstr — no mismatched precision.

4. Flow of forms

I like your idea:

- **Onboarding Intake Form** → captures initial swaps and reserves.
- **Order Input Form** → logs actual Lobstr order details (Active Orders).
- **CycleSet Log** → auto-updates based on those inputs, calculates profits, next order prices, etc.
- Optional: **Profit Summary Tab** → pretty, user-friendly output with "Next Trade Instructions" phrased as sentences.

That way Incredible doesn't need to scroll through columns of raw data. He just checks one tab that says:

👉 "Next Trade: Sell 1,000 AFR @ 0.0013000 USDC for 1.3000 USDC."

💡 Question for you: do you want me to **mock up the Order Input Form** next (so you can see how clean entry + ROUNDing works), or should I wait until you've finished polishing the Onboarding Intake Form?

You said:

Boo Boo 🤔👉 I want you to check some code for me and see if you can figure out why it doesn't work the way the article says it should work. I created the script, but when I add the placeholder text it disappears instead of waiting for data to be entered. Here's the instructions that were in the article: Creating a Simple Script Here's a basic example to get you started: `function onEdit(e) { var sheet = e.source.getActiveSheet(); var cell = e.range; if (cell.getValue() === 'Enter data here') { cell.setValue(""); } }` To use this script: Go to Extensions > Apps Script. Delete any code in the editor and paste the above script. Click the disk icon to save, then close the script editor. With this script, any cell containing the text "Enter data here" will be cleared once the user starts editing it. You can customize the placeholder text to suit your needs.

ChatGPT said:

Ah, I see what's happening Boo Boo 🤔👉 The article is a little misleading. Let me break it down:

The script you pasted:

```
function onEdit(e) {  
  var sheet = e.source.getActiveSheet();  
  var cell = e.range;  
  
  if (cell.getValue() === 'Enter data here') {  
    cell.setValue('');  
  }  
}
```

What it actually does:

- The `onEdit` trigger only fires *after* you change a cell.
- But the `if` is checking if the cell's value **is already** "Enter data here" at the moment of edit.
- Since the act of editing replaces "Enter data here" immediately, the condition almost never matches. That's why it looks like the placeholder just disappears instead of waiting.

✅ **Solution:**

You want the placeholder to disappear the *moment the user starts typing*. For that, you check the **old value** (`e.oldValue`) instead of the new one. Try this:

```
function onEdit(e) {  
  var cell = e.range;  
  var oldValue = e.oldValue;  
  
  if (oldValue === 'Enter data here') {
```

```
    // Do nothing, user overwrote it
  }
}
```

Or, if you want to actively clear the placeholder when the user touches it (even if they didn't type anything yet), you can do:

```
function onEdit(e) {
  var cell = e.range;

  if (cell.getDisplayValue() === 'Enter data here') {
    cell.setValue('');
  }
}
```

⚠ One catch:

Google Sheets **does not natively support true placeholder text** (like a web form). The trick most people use is:

1. Pre-fill the cell with “Enter data here”.
2. Use conditional formatting to make that text gray/italic so it *looks* like a placeholder.
3. Then use a script like the one above to wipe it the moment they type.

Do you want me to rewrite this into a clean version that:

- makes the placeholder gray/italic,
- wipes it when touched,
- and only works for specific columns (like input fields)?

You said:

Hmmmm Poof 🧙🏽💖 well what I really want it to do (and probably can't) is have a grey text of 7 pt font that looks like a placeholder that disappears once new data is entered not just when the field is touched or selected. But I want the data entered to be black and 11 pt. I also want to explore a function to clear fields so that a data entry form can be reused like when entering data for new orders or new deposits which would not use the onboarding intake form. I would want the data entered to go to the next available cell in the designated column of another sheet rather than a specific cell. I know this is kind of complex, but I am trying to think of workarounds.

ChatGPT said:

Got it Boo Boo 🧙🏽💖 — you're running into one of those “Sheets is not a web app” limitations. Let me unpack both of your goals:

1. True placeholder behavior (gray, small font, disappears on entry)

Unfortunately, Google Sheets doesn't support *true placeholders* the way web forms do. But we can mimic it with a combo of tricks:

- **Step 1:** Pre-fill cells with "Enter data here".
- **Step 2:** Use **conditional formatting**:
 - Rule: `=A1="Enter data here"` (adjust for your range).
 - Format: gray, 7pt italic.
- **Step 3:** Script clears placeholder only if new text is entered.

Here's a script that does that:

```
function onEdit(e) {  
  var cell = e.range;  
  var value = e.value;  
  var oldValue = e.oldValue;  
  
  // Only clear if old value was the placeholder AND new value is not empty  
  if (oldValue === "Enter data here" && value !== "") {  
    // Set the new value as is, then format it black 11pt  
    cell.setFontColor("black").setFontSize(11);  
  }  
}
```

👉 The conditional formatting makes the placeholder *look* gray + small, and the script makes the real input black + 11pt.

2. Reusable data entry form (clearing fields + appending rows elsewhere)

Yes, this is doable. Instead of writing directly to a specific cell, the script can “append” to the next empty row in another sheet. Here's a template:

```
function submitData() {  
  var ss = SpreadsheetApp.getActiveSpreadsheet();  
  var formSheet = ss.getSheetByName("Order Entry Form"); // your input sheet  
  var logSheet = ss.getSheetByName("Orders Log"); // destination sheet  
  
  // Example: read data from cells B2:D2  
  var data = formSheet.getRange("B2:D2").getValues()[0];  
  
  // Append to the next available row in log sheet  
  logSheet.appendRow(data);  
  
  // Clear the entry form fields after submission  
  formSheet.getRange("B2:D2").clearContent();  
}
```

You can assign this `submitData()` function to a button in your form tab. Then you type → hit submit → the row gets added to your log → form clears itself.

🌟 So in short:

- Placeholder can be faked (gray text → replaced with black input).
- Reusable forms are absolutely doable (data entry → append → clear).

Would you like me to **mock up both behaviors in one coherent script** (placeholder + reusable form submit/clear) so you've got a single framework to adapt?

You said:

Boo Boo 💖🌟🤖 I already saved the first placeholder script as "Enter Amount" and changed the placeholder text to "Enter Amount". I could make custom placeholders that say more specifically what to enter, and you could put all of the ones I would use in one script as well as the reusable forms script. The only problem is that they don't have buttons like they have in forms, so how would we make a button to assign the `submitData()` function to?

ChatGPT said:

Ahhh gotcha Boo Boo 💖🌟🤖 — you're right: Google Sheets doesn't have native "Form buttons" like Google Forms, but we can fake it with **drawing objects** or **shapes**. Here's how you can make a "Submit" button for your `submitData()` script:

✅ Adding a Button in Google Sheets

1. Go to **Insert > Drawing > + New**.
2. Create a simple rectangle (or any shape) and type "Submit" inside.
3. Save and close → the button will appear in your sheet.
4. Right-click the button → **Assign script**.
5. Type the function name exactly:
`submitData`
6. Now when you click the button, it runs your script → appends the row → clears the fields. 🌟🤖

📄 Combining multiple placeholder rules

Yes, we can handle all placeholders in a single script. Example:

```
function onEdit(e) {  
  var cell = e.range;  
  var oldValue = e.oldValue;  
  var value = e.value;
```

```
// List of placeholders
var placeholders = ["Enter Amount", "Enter Price", "Enter Asset", "Enter Date"];

// If old value was a placeholder and new value is not empty
if (placeholders.indexOf(oldValue) !== -1 && value !== "") {
  cell.setFontColor("black").setFontSize(11);
}
}
```

Now any cell starting with one of those placeholders will flip to black 11pt text once edited.

⚡ Final combo plan

- **Onboarding Intake Form tab** → uses placeholders for input cells.
 - **Submit button** (drawing object) → runs `submitData()`.
 - **Script** → sends data to the right log tab + clears placeholders for reuse.
-

Do you want me to stitch these two scripts (placeholders + submit/clear) into **one integrated script** so you won't have to juggle them separately?

Yes Boo Boo 🧡🌟🐼— you don't need two separate scripts. I can merge the **placeholder logic** and the **submit/clear form logic** into one neat script. That way:

- Placeholders (gray/7pt) disappear and convert to black/11pt when the user types.
- A **Submit** button (made with a drawing shape in Sheets) runs the function to:
 - take the entered data,
 - append it to the right log sheet,
 - reset the entry form fields back to placeholders for reuse.

Here's a combined version:

```
function onEdit(e) {
  var cell = e.range;
  var oldValue = e.oldValue;
  var value = e.value;

  // List of placeholder texts
  var placeholders = ["Enter Amount", "Enter Price", "Enter Asset", "Enter Date"];

  // If user replaces a placeholder with something else
  if (placeholders.indexOf(oldValue) !== -1 && value !== "") {
    cell.setFontColor("black").setFontSize(11);
  }
}

// Submit + clear function
function submitData() {
  var ss = SpreadsheetApp.getActiveSpreadsheet();
```

```

var formSheet = ss.getSheetByName("Onboarding Intake Form"); // input sheet
var logSheet = ss.getSheetByName("Onboarding"); // destination log sheet

// Example: assume form fields are in cells B2:E2
var formRange = formSheet.getRange("B2:E2");
var data = formRange.getValues()[0];

// Append to next available row in log sheet
logSheet.appendRow(data);

// Reset form fields back to placeholders
var placeholders = ["Enter Amount", "Enter Price", "Enter Asset", "Enter Date"];
for (var i = 0; i < placeholders.length; i++) {
  formSheet.getRange(2, i +
2).setValue(placeholders[i]).setFontColor("gray").setFontSize(7);
}
}

```

How to hook it up

1. Put placeholders in row 2 of your **Onboarding Intake Form** sheet.
 2. Insert a drawing → make it say “Submit”.
 3. Right-click → **Assign script** → type `submitData`.
 4. When clicked, it saves the row to **Onboarding** and resets the intake form with placeholders.
-

Would you like me to also show you how to make the **reset part dynamic** (so you can add or remove form fields later without rewriting the script)?

You said:

Hey Poof 🧡🌟🤖 yesterday because of ChatGPT limits on code you were not allowed to respond to the following message I sent, but luckily I copied it before hitting the "Retry" button today, because it erased the message. Anyway, here is the pasted message: "Ok Boo Boo 🧡🌟🤖 this is great stuff. I'm so excited! I think what we should do is plan to write a single script that applies to both of the PortalX LITE Google Sheets files (basic and advanced) that I'll call "PortalX LITE Forms Logic" or something depending on what all ends up in it. We may end up coming up with other functions later. But if you'd like to start making an integrated script that I can edit as I go along, then you can do that now I guess. But before you do that, I need to better understand the logic for getting data from a form and inputting that data into the right fields on the receiving log sheet. These are not necessarily going to be all in a row in a range like "B2:D2". Each cell with data in the form sheet will be inputting data into cells in various columns of sometimes more than one other sheet." Also, I heard back from Incredible. He was not able to open the .xlsx file I sent him to get feedback on whether his TalkBack could read anything in it. He said he was not bale to though I had realized after I sent it to him that when you click on an Excel file in Telegram from a mobile device it does not take you automatically to Google Sheets or any other Excel compatible app like MS Copilot. It only opens a read only file that you can't do anything

with. You can select anything or anything. Incredible also told me that he has not been feeling well and has developed some back pain and is feeling a sense of disappointment that he is not able to do the kind of content that he was doing before for this project. You see I had asked Incredible to send me some of his previous content but he has lost access to his accounts when he switched devices, and he doesn't have someone who can help him all of the time, but I was able to easily find his "Incredible Master TV" channel on Facebook by simply searching "Incredible Master" the name he uses on Telegram. The content he used to do was to repost videos related to African life, pop culture and politics. a totally different energetic resonance than what we are doing with EH 2.0 and EA 2.0. I had also mentioned to him that the order he was waiting to fill should have filled because AFR had a price spike which normally happens when XLM has a price spike. Incredible made a comment that this may be a sign to get into trading and mentioned something about maybe getting another 200 dollars worth of AFR, so clearly he doesn't get the concepts we are teaching yet. Anyway, I wrote him the following long message: "No Incredible your thinking about trading is not correct. The AFR market will remain unpredictable until we create a stable market with what we are doing. Just because the price goes up is not an indicator that it is a good market to trade in. In fact the better time for people to enter the market is when the price is down. Markets are only good for day trading if they steadily go up and down which markets in Stellar do not do. What we are teaching people is not day trading. We are teaching people basic digital asset management for crypto assets that don't act predictably. This trading logic also works for day trading and will be used for the eventual automated PortalX system, but that is not where we are right now. I understand that you are very attached to the work you used to do, but this is a resonance test for you right now. I am offering you the opportunity of a lifetime to do something truly world changing, but it is not something that will happen instantly. Unfortunately, I am not in the position to be able to fund what you are talking about. I know you haven't had a chance to really study all that I present in Elevate Humanity, but it is about making an energetic shift from the Earth simulation to the New Earth layers of reality. That requires shifting energetically to the frequency of divine sovereignty. These are metaphysical truths that most people in the world are completely oblivious to. In order to make such a shift, people need to be gradually spoon fed information that will start shifting them from simulation controlled systems to sovereign systems. The Stellar network as a sovereign alternative to simulation based financial systems is a first non-metaphysical step towards sovereignty. Things like pop culture and politics are things that the design of the Earth simulation uses to create duality. But Divine Truth is unity. I'm not telling you to give up on making content about African culture, but I'm asking you to take a break from it and help me create something that will eventually let you re-emerge as a more empowering influencer. As far as finding your channel on Facebook. That was simple. Your name on this Telegram account is "Incredible Master", so I just searched it and found Incredible Master TV and saw that you also had Instagram and TikTok accounts of the same name which I did not go look at because I got the information I was looking for. You have to understand that when you are on the internet anyone can find you and find out all kinds of information about you. Anyway, I want to talk to you about resonance. You will notice in Elevate Humanity that I talk about resonance a lot, because resonance is what creates the reality we will experience, and it is what dictates what is commonly known as "The Law of Attraction", but most people do not understand the metaphysical science that explains the phenomenon. Nikola Tesla understood these things, and that is why I quote him in my teachings. The truth is that everything is